# Reed–Muller Encoding Leakage Enables Single-Trace Message Recovery in HQC

Jaeho Jeon[1], Donghyen Kim[1], Suseong Lee[1] and Young-Sik Kim[1]

Department of Electrical Engineering and Computer Science, DGIST, Daegu, Korea,
dgwogh@dgist.ac.kr,dhkim200426@dgist.ac.kr,mercury@dgist.ac.kr,ysk@dgist.ac.kr

**Abstract.** HQC is a code-based key-encapsulation mechanism standardized by NIST, whose decapsulation follows a Fujisaki–Okamoto (FO) transform and therefore re-executes encryption-side encoding during deterministic re-encryption. In this paper, we show that this design choice exposes a critical leakage point in the *Reed–Muller (RM) encoding* routine: across the NIST-submitted implementations, the HQC team's official codebase, and the PQClean implementations.

We demonstrate the practical impact of this leakage on a ChipWhisperer CW308 UFO board with an STM32F303 (Cortex-M4) target. Using a total of 5,000 power traces for profiling and evaluation, we recover the full 128-bit encapsulation message from a *single* decapsulation trace with up to 96.9% success. In comparison, the current state of the art for single-trace HQC message recovery based on *soft-analytical side-channel attacks* (SASCA) reports profiling on the order of 500,000 traces; our approach therefore reduces the required profiling budget by two orders of magnitude while achieving comparable single-trace capability.

Beyond session-key compromise, we show that direct recovery of the decrypted message can serve as an oracle primitive that substantially lowers the cost of oracle instantiation in prior HQC secret-key recovery frameworks. While prior oracle instantiations typically map leakage to a discrete set of task-specific labels, our approach recovers the decrypted message itself, and thus applies uniformly over the full message space (i.e., arbitrary $m'$ values). Concretely, we reduce the profiling cost required to instantiate a *decryption success/failure* oracle, multi-value plaintext-checking, and full-decryption oracles by approximately 90.3%, 84.83%, and 26.7%, respectively.

**Keywords:** HQC · Hamming Quasi-Cyclic · PQC · Post-Quantum Cryptography · KEM · Embedded systems · side-channel attacks

## 1 Introduction

The advancement of quantum computing presents a significant threat to classical public-key cryptographic systems such as RSA [RSA78] and ECC [Mil86]. In particular, Shor's algorithm [Sho94] has demonstrated the ability to efficiently break these schemes in polynomial time on a quantum computer. In response, the US National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) [BL17] standardization process [BL17, CCJ+16]. As a result, in 2022, several PQC schemes, such as the lattice-based key encapsulation mechanism (KEM) CRYSTALS-Kyber [BDK+18], the lattice-based signature schemes CRYSTALS-Dilithium [DKL+18] and Falcon [FHK+18], and the stateless hash-based signature scheme SPHINCS+ [BHK+19], were selected as new NIST PQC standards. Recognizing that diversified assumptions about computational hardness are essential for robust cryptographic security, the standardization process emphasized the inclusion of schemes based on diverse mathematical foundations [AAC+22]. Accordingly,

the fourth-round KEM finalists featured code-based candidates such as BIKE [ABB+22], Classic McEliece [BCL+17], and Hamming Quasi-Cyclic (HQC) [MAB+18], with HQC officially adopted as a new standard in 2025 [ABC+25].

Beyond theoretical robustness, NIST has underscored the importance of practical security in real deployments, including embedded and resource-constrained platforms [AAC+22]. In such settings, side-channel leakage through power, electromagnetic emanations, or timing variations can undermine the security of otherwise sound cryptographic designs. Consequently, implementation security—and in particular resistance against side-channel attacks—is a critical requirement for post-quantum cryptography intended for widespread adoption.

## 1.1 Message and Ephemeral Secret Recovery in PQC

While physical attacks on PQC have traditionally focused on recovering *long-term secret keys*, prior work shows that *ephemeral secrets*—most notably the encapsulation/encryption message—can be equally damaging when implementations leak during encoding or decoding. In lattice-based KEMs, such leakage enables *single-trace message recovery* (e.g., Kyber and Saber), and in KEMs built via the Fujisaki–Okamoto transform (FO transform) [FO99] the recovered message often suffices to recompute the shared session key from public transcript data [SKL+20]. Ravi *et al.* further systematized this threat across NIST PQC candidates, showing that even *partial* message recovery can substantially simplify follow-up attacks, including chosen-ciphertext-assisted recovery of long-term secrets [RBRC21].

Message/plaintext recovery is also practical in code-based cryptography. Lahr *et al.* demonstrate partial plaintext recovery for Classic McEliece via side-channel-assisted information-set decoding [LNPS20], and related profiled attacks confirm that message-related leakage can be exploited in syndrome-decoding settings [CDCG22]. For HQC, Goy *et al.* achieve *single-trace* shared-key recovery using SASCA methods [VCGS14, GMGL24] by exploiting leakage from Reed–Solomon finite-field multiplication (e.g., `gf_mul`) and performing probabilistic inference. Moreover, message recovery can persist even under countermeasures: Ngo *et al.* recover the encapsulation message from a *first-order masked, IND-CCA-secure* Saber implementation using deep-learning-based analysis, and leverage it toward long-term key recovery [NDGJ21]; similar encoding/decoding-dependent leakage has also been analyzed for FrodoKEM [Ber25].

These results are particularly relevant in the KEM setting because FO-style transforms re-execute encryption-side computations during decapsulation via deterministic re-encryption. Consequently, message-dependent encoding/decoding routines may leak even when the adversary can only observe decapsulation [FO99, RBRC21, Ber25]. This observation motivates revisiting HQC with a focus on message-dependent leakage in its encoding pipeline.

## 1.2 Oracle-Based Side-Channel Attacks on HQC

During the NIST PQC process, HQC has become a prominent target for oracle-based side-channel cryptanalysis. A series of works progressively refined how to construct and exploit plaintext-checking and decryption oracles, leading to increasingly efficient full secret-key recovery attacks.

Early practical attacks exploited cache-timing leakage to instantiate a plaintext-checking (PC) oracle that distinguishes valid from invalid decryptions, showing that even ostensibly constant-time implementations may leak enough information to bypass CCA security in a chosen-ciphertext setting [HSC+23]. Subsequent work demonstrated that the oracle abstraction is largely agnostic to the leakage source: the divide-and-surrender attack leveraged subtle instruction-level timing variations in arithmetic operations to build higher-accuracy PC oracles and to reduce the number of required oracle queries well beyond

earlier cache-based approaches [SGG24].

More recently, the emphasis shifted from *finding* new leakage sources to maximizing the *information extracted per oracle call*. OT-PCA introduced an offline-template approach that derives soft information from Reed–Muller decoding outcomes, enabling full key recovery with orders of magnitude fewer oracle calls even under imperfect oracle accuracy [DG25b]. In parallel, multi-value plaintext-checking and full-decryption oracles further generalized the oracle model so that each query can convey richer information about multiple secret positions simultaneously [DG25a].

Overall, oracle-based attacks on HQC have developed into a theoretically mature and highly effective framework: the oracle models have been systematically refined, and the resulting key-recovery methods achieve markedly improved efficiency.

## 1.3 Motivation and Key Observation

The preceding discussion highlights two complementary trends in the side-channel analysis of post-quantum cryptography. On the one hand, message and ephemeral-secret recovery attacks show that leakage in encoding/decoding can directly compromise session keys in KEM constructions. On the other hand, oracle-based attacks on HQC have developed into a powerful framework for long-term secret-key recovery once a suitable leakage-induced oracle can be instantiated.

**Message recovery in HQC: known, but not yet efficient.** Message recovery in HQC has recently been demonstrated: Goy *et al.* achieve single-trace shared-key recovery using SASCA by exploiting leakage in the Reed–Solomon finite-field multiplication routine (e.g., `gf_mul`) and performing belief-propagation-based inference [GMGL24]. This confirms that message-dependent leakage can be sufficient to compromise the derived key. However, the SASCA route relies on substantial profiling and nontrivial inference to translate noisy leakage into message bytes.

**Beyond session keys: message recovery as a flexible oracle primitive.** In this work, we identify the Reed–Muller (RM) encoding routine (RM(1,7) in HQC) as an even more critical leakage source [MAB+18, MS77]. In contrast to leakage that requires heavy post-processing, the RM encoder exhibits highly structured, bit-level data dependence due to bit-to-word mask expansions in the optimized implementation. This yields strong and temporally separable features that allow recovering the encapsulation message with substantially lower analysis complexity, enabling practical single-trace message (and hence session-key) recovery.

**Why this matters beyond session-key compromise: enabling cheaper oracle instantiation.** The same primitive also strengthens oracle-based key-recovery attacks on HQC. Existing PC/MV-PC/FD frameworks ultimately derive their oracle outputs from the (decrypted) message value used in FO-style deterministic re-encryption, and instantiating these oracles typically requires extensive profiling for multi-class leakage models and carefully engineered classifiers [DG25b, DG25a]. Once the message is recovered directly from RM-encoding leakage, the attacker can compute PC/MV-PC labels locally and thereby simplify (and in parts replace) the oracle-instantiation step. This connects efficient message recovery to more practical oracle-based secret-key recovery.

## 1.4 Our Contributions

Our findings and attacks apply to the HQC team's latest upstream implementation, the NIST fourth-round optimized implementation, and the corresponding PQClean implementations on Cortex-M4-class microcontrollers, and persist under common embedded build

settings (both `-O3` and `-Os`). We show that this encoding-side leakage enables efficient recovery of the encapsulation message, immediately compromising the derived session key, and further serves as a practical primitive to simplify oracle instantiation in existing HQC key-recovery frameworks. Our main contributions are as follows.

- **Single-trace message and session-key recovery from RM-encoding leakage.** We identify strong, structured leakage in the RM(1,7) encoder of HQC caused by message-bit expansion effects that yield temporally separable, message-dependent leakage points on Cortex-M4 microcontrollers. Leveraging this separation, we recover the full 128-bit encapsulation message from a *single* decapsulation trace and recompute the corresponding session key from public transcript information, achieving up to **96.9%** full-message recovery with **5,000** traces in total (**4,000** for profiling and **1,000** for evaluation). Compared to prior single-trace HQC message-recovery results [GMGL24] that rely on SASCA and report profiling on the order of 500,000 traces, our approach reduces the profiling requirement by approximately two orders of magnitude. The same leakage source is also present on the encryption side whenever an adversary can measure executions of HQC-PKE.Encrypt.

- **Message recovery as an oracle primitive that simplifies HQC key recovery.** We demonstrate that direct recovery of the decrypted message yields an oracle primitive that substantially simplifies oracle-based key recovery on HQC. Whereas prior PC/MV-PC/FD [DG25b, DG25a] instantiations typically classify leakage into a fixed set of pre-trained labels, our method recovers the message itself, avoiding any restriction to pre-defined classes and covering the full $2^{128}$ message space. This enables computing PC/MV-PC outcomes locally and partially replacing FD components associated with the message region. Concretely, this reduces the profiling cost for instantiating a decryption success/failure oracle, multi-value plaintext-checking, and full-decryption oracles by approximately **90.3%**, **84.83%**, and **26.7%**, respectively.

- **Reproducibility.** To support follow-up research and facilitate reuse of our experimental methodology, we provide our source code and experimental artifacts as anonymous submission material for reviewers, and we will release them publicly in a GitHub repository upon publication.

# 2 Preliminaries

## 2.1 Overview of HQC

We summarize the HQC-PKE algorithms at a high level, omitting implementation details related to XOF initialization, seed expansion, and the implementation-specific compression/truncation of ciphertext components. We focus instead on the algebraic structure relevant to oracle construction and message leakage.

**Key generation.** HQC-PKE key generation samples two sparse secret vectors $x, y$ and a dense public vector $h$, and computes $s = x + h \cdot y$. The public key is $\mathsf{pk} = (h, s)$ and the secret key contains $\mathsf{sk} = (x, y)$ (and associated seeds in the concrete implementation).

**Algorithm 1** HQC-PKE.Keygen (simplified)

**Require:** Parameters $(\mathcal{R}, \mathcal{R}_\omega)$
**Ensure:** Public key $\mathsf{pk} = (h, s)$ and secret key $\mathsf{sk} = (x, y)$
1: Sample $x \leftarrow \mathsf{SampleFixedWeightVect}(\mathcal{R}_\omega)$ ▷ $x$ sparse
2: Sample $y \leftarrow \mathsf{SampleFixedWeightVect}(\mathcal{R}_\omega)$ ▷ $y$ sparse
3: Sample $h \leftarrow \mathsf{SampleVect}(\mathcal{R})$ ▷ $h$ dense
4: $s \leftarrow x + h \cdot y$
5: **return** $\mathsf{pk} = (h, s)$, $\mathsf{sk} = (x, y)$

**Encryption.** To encrypt a message $m$ under $\mathsf{pk} = (h, s)$, the algorithm samples sparse ephemeral values $(r_1, r_2, e)$ and outputs a ciphertext $c_{\mathrm{PKE}} = (u, v)$ where

$$u = r_1 + h \cdot r_2, \qquad v = \mathsf{C.Encode}(m) + s \cdot r_2 + e.$$

Here, $\mathsf{C.Encode}(\cdot)$ denotes HQC's concatenated RS/RM encoding, whose implementation is analyzed in Section 3.

**Algorithm 2** HQC-PKE.Encrypt (simplified)

**Require:** Public key $\mathsf{pk} = (h, s)$, message $m$
**Ensure:** Ciphertext $c_{\mathrm{PKE}} = (u, v)$
1: Sample $r_1 \leftarrow \mathsf{SampleFixedWeightVect}(\mathcal{R}_\omega)$ ▷ $r_1$ sparse
2: Sample $r_2 \leftarrow \mathsf{SampleFixedWeightVect}(\mathcal{R}_\omega)$ ▷ $r_2$ sparse
3: Sample $e \leftarrow \mathsf{SampleFixedWeightVect}(\mathcal{R}_\omega)$ ▷ $e$ sparse
4: $u \leftarrow r_1 + h \cdot r_2$
5: $v \leftarrow \mathsf{C.Encode}(m) + s \cdot r_2 + e$
6: **return** $c_{\mathrm{PKE}} = (u, v)$

**Decryption.** Given $c_{\mathrm{PKE}} = (u, v)$ and secret $y$, decryption computes

$$m = \mathsf{C.Decode}\big(v - u \cdot y\big),$$

where $\mathsf{C.Decode}(\cdot)$ denotes HQC's concatenated RS/RM decoding. This affine structure is central to *oracle-based* attacks on HQC: under chosen-ciphertext access, an adversary can craft $(u, v)$ so as to control (or bias) the decoder input $v - u \cdot y$, and then infer information from distinguishable decoding behavior, yielding plaintext-checking or decryption oracles [DG25b, DG25a].

**Algorithm 3** HQC-PKE.Decrypt (simplified)

**Require:** Secret key $\mathsf{sk}$ containing $y$, ciphertext $c_{\mathrm{PKE}} = (u, v)$
**Ensure:** Message $m$
1: $m \leftarrow \mathsf{C.Decode}\big(v - u \cdot y\big)$
2: **return** $m$

**KEM decapsulation and FO re-encryption.** HQC-KEM applies an FO-style transform on top of HQC-PKE. Decapsulation first computes a candidate message $m' \leftarrow \mathsf{HQC\text{-}PKE.Decrypt}(\mathsf{sk}_{\mathrm{PKE}}, c_{\mathrm{PKE}})$, then deterministically derives both the candidate shared key and the re-encryption randomness as $(K', \theta') \leftarrow G(H(\mathsf{pk}_{\mathrm{KEM}}) \,\|\, m' \,\|\, \mathsf{salt})$, and re-encrypts $m'$ as $c'_{\mathrm{PKE}} \leftarrow \mathsf{HQC\text{-}PKE.Encrypt}(\mathsf{pk}_{\mathrm{KEM}}, m', \theta')$ to validate the ciphertext. If $c'_{\mathrm{PKE}} = c_{\mathrm{PKE}}$ (and $m' \neq \perp$), the output is $K = K'$; otherwise the implementation returns a rejection key $\bar{K} \leftarrow J(H(\mathsf{pk}_{\mathrm{KEM}}) \,\|\, \sigma \,\|\, c_{\mathrm{KEM}})$. Crucially, the deterministic re-encryption re-executes $\mathsf{C.Encode}(m')$, which is the leakage point studied in this work.

**Algorithm 4** HQC-KEM.Decaps (FO transform, simplified)

**Require:** Secret key $\mathsf{sk}_{\mathrm{KEM}}$ (contains $\mathsf{sk}_{\mathrm{PKE}}$, $\mathsf{pk}_{\mathrm{KEM}}$, and $\sigma$), ciphertext $c_{\mathrm{KEM}}$ (contains $c_{\mathrm{PKE}}$ and $\mathsf{salt}$)
**Ensure:** Shared key $K$

$m' \leftarrow \mathsf{HQC\text{-}PKE.Decrypt}(\mathsf{sk}_{\mathrm{PKE}}, c_{\mathrm{PKE}})$
$(K', \theta') \leftarrow G\big(H(\mathsf{pk}_{\mathrm{KEM}}) \,\|\, m' \,\|\, \mathsf{salt}\big)$
$c'_{\mathrm{PKE}} \leftarrow \mathsf{HQC\text{-}PKE.Encrypt}(\mathsf{pk}_{\mathrm{KEM}}, m', \theta')$ ▷ re-encryption
$\bar{K} \leftarrow J\big(H(\mathsf{pk}_{\mathrm{KEM}}) \,\|\, \sigma \,\|\, c_{\mathrm{KEM}}\big)$ ▷ rejection key
**if** $m' \neq \perp$ **and** $c'_{\mathrm{PKE}} = c_{\mathrm{PKE}}$ **then**
    $K \leftarrow K'$
**else**
    $K \leftarrow \bar{K}$
**end if**
**return** $K$

The highlighted re-encryption step re-executes $\mathsf{C.Encode}(m')$ during decapsulation, providing the observation point for both our message-recovery attack and the oracle instantiation discussed later.

## 2.2 Concatenated RS/RM Encoding and Decoding in HQC (HQC-128)

As introduced in Section 2.1, HQC encodes the 128-bit encapsulation/encryption message $\mathbf{m}$ using a concatenated error-correcting code $\mathsf{C.Encode}(\cdot)$: an outer Reed–Solomon (RS) code over $\mathbb{F}_{256}$ followed by an inner (binary) Reed–Muller (RM) code (Fig. 1) [MAB+18]. Intuitively, the RS layer protects the message at the *symbol* (byte) level, while the RM layer protects each RS symbol at the *bit* level before transmission.

**Encoding: RS then RM (with duplication).** For HQC-128, the RS encoder takes the 128-bit message $m$ (i.e., $k = 16$ bytes) and produces an RS codeword of length $n_1 = 46$ bytes, expanding the payload from $16 \times 8 = 128$ bits to $46 \times 8 = 368$ bits. Moreover, HQC uses a *systematic* RS encoder, meaning the message bytes appear *verbatim* in fixed positions of the RS codeword:

$$\texttt{cdw}[30..45] = m[0..15], \qquad \texttt{cdw}[0..29] \text{ are parity symbols.}$$

In other words, the red-boxed region in Fig. 1 corresponds exactly to the systematic part of the RS codeword that directly contains $m$.

After RS encoding, HQC applies the inner RM encoder independently to each RS symbol. Concretely, HQC uses the first-order RM code $\mathrm{RM}(1, 7)$, which maps one 8-bit input symbol to a 128-bit binary codeword. HQC then applies a duplication factor $d = 3$ (bit repetition), so each RS symbol becomes a $128 \cdot 3 = 384$-bit block. Thus, encoding the full RS codeword of 46 symbols yields a total of

$$46 \times 384 = 17{,}664 \text{ bits},$$

which naturally decomposes into a parity block ($30 \times 384 = 11{,}520$ bits) and a message block ($16 \times 384 = 6{,}144$ bits), mirroring the systematic RS partition.

**From encoding to encryption noise.** In HQC-PKE encryption, the binary codeword $\mathsf{C.Encode}(m)$ is then combined with ciphertext-dependent terms and sparse noise (error injection). As a result, the receiver observes a noisy version of the concatenated codeword, which must be corrected by $\mathsf{C.Decode}(\cdot)$.

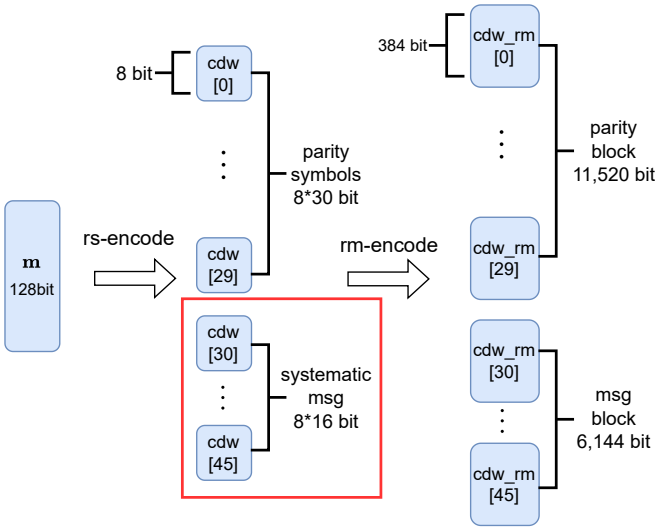


**Figure 1:** Concatenated RS/RM encoding in HQC (HQC-128 parameters). A 128-bit message $m$ is first encoded by a systematic Reed–Solomon code to form a 46-byte RS codeword $\texttt{cdw}[0..45]$, where $\texttt{cdw}[0..29]$ are parity symbols and $\texttt{cdw}[30..45]$ copy the message bytes verbatim. Each 8-bit RS symbol is then independently Reed–Muller encoded (RM$(1,7)$) and duplicated ($d = 3$), producing a 384-bit block per symbol. Concatenating all 46 blocks yields $46 \times 384 = 17{,}664$ encoded bits, which can be viewed as a parity block (11,520 bits) and a message block (6,144 bits) corresponding to the systematic region.

**Decoding: RM block correction then RS symbol correction.** Decoding proceeds in the reverse order. First, RM decoding is applied *block-wise* to each duplicated RM chunk $\texttt{cdw\_rm}[i]$ for $i \in \{0, \ldots, 45\}$. HQC uses the first-order Reed–Muller code RM$(1,7)$ with parameters $[128, 8, 64]$, and then duplicates each bit $d$ times (for HQC-128, $d = 3$), yielding a duplicated RM code of parameters $[384, 8, 192]$ (minimum distance $d_{\min} = 192$). In particular, in the standard bounded-distance view, a length-384 RM block can correct up to $\lfloor (d_{\min} - 1)/2 \rfloor = 95$ bit errors within the block. Operationally, each block is decoded via a Hadamard-transform-based procedure adapted to duplication, outputting an 8-bit estimate of the corresponding RS symbol. Depending on the noise realization, an RM block may decode successfully (yielding a reliable symbol) or fail/produce a noisy estimate.

Second, RS decoding operates on the recovered 46-byte symbol sequence and corrects remaining errors at the *symbol* level. In HQC-128, the outer code is the shortened, systematic Reed–Solomon code RS-S1 with parameters $(n_1, k, d_{\min}) = (46, 16, 31)$. Equivalently, its correcting capacity is $\delta = 15$ (since $n_1 - k = 2\delta$), and shortening does not change this capability. Accordingly, the RS decoder can correct up to $\delta = 15$ erroneous RS symbols across the 46-symbol word, after which the original message $m$ is read from the systematic part of the corrected RS codeword.

# 3 Message Leakage via Reed–Muller Encoding in HQC

This section explains leakage sources arising from the Reed–Muller (RM) encoding routine in HQC and shows how the leakage enables recovery of message-dependent information from a single execution trace.

Our analysis targets the official HQC reference implementation released by the HQC team, specifically the GitLab upstream version tagged `v5.0.0` (Aug. 22, 2025) and the subsequent commit `d622142a` (Aug. 26, 2025), available from their public GitLab repository

(`https://gitlab.com/pqc-hqc/hqc/`).[GAMA+25, Tea26] Unless stated otherwise, all code-level observations and implementation details reported in this section refer to that codebase.

Our analysis targets the upstream HQC-team reference implementation. We note, however, that its RM-encoding structure is preserved in the NIST-submitted reference and `optimized_implementation` as well as in the corresponding PQClean implementations. Therefore, the same vulnerable behavior (and hence the attack surface) carries over to these codebases.

## 3.1 Re-encryption and Message Encoding in HQC

HQC decapsulation follows a Fujisaki–Okamoto (FO) style transform: it first decrypts a PKE ciphertext to obtain a candidate message $m'$, and then deterministically *re-encrypts* $m'$ to verify ciphertext validity (Algorithm 4):

$$m' \leftarrow \mathsf{HQC\text{-}PKE.Decrypt}(\mathsf{dk}_{\mathsf{PKE}}, c_{\mathsf{PKE}}), \qquad c'_{\mathsf{PKE}} \leftarrow \mathsf{HQC\text{-}PKE.Encrypt}(\mathsf{ek}_{\mathsf{PKE}}, m', \theta').$$

This FO re-encryption re-executes the encryption-side message encoding $\mathsf{C.Encode}(m')$ (RS then RM with duplication; see Section 2.2), so message-dependent leakage in the encoding pipeline is observable even when the adversary can only measure decapsulation.

As a consequence, a key feature of HQC becomes particularly relevant: the RS layer is implemented in *systematic* form, where the message symbols are copied verbatim into fixed positions of the RS codeword, while the remaining symbols carry parity (Fig. 1). In particular, for HQC-128 the RS output satisfies $\mathtt{cdw}[30..45] = m[0..15]$, so the subsequent RM encoder processes these message bytes *as part of* its per-symbol inputs when producing the corresponding RM blocks. For our analysis, this creates a simple and fixed correspondence between message bytes and a contiguous region of RS symbols that is immediately processed by RM encoding.

In the HQC reference code, RS encoding is implemented as follows:

Listing 1: Systematic Reed–Solomon encoding in HQC (excerpt).

```
void reed_solomon_encode(uint64_t *cdw, const uint64_t *msg) {
    size_t i, j, k;
    uint8_t gate_value = 0;

    uint16_t tmp[PARAM_G] = {0};
    uint16_t PARAM_RS_POLY[] = {RS_POLY_COEFS};

    uint8_t msg_bytes[PARAM_K] = {0};
    uint8_t cdw_bytes[PARAM_N1] = {0};

    memcpy(msg_bytes, msg, PARAM_K);

    /* RS parity computation: fills cdw_bytes[0 .. PARAM_N1-PARAM_K-1] (
        omitted) */

    memcpy(cdw_bytes + PARAM_N1 - PARAM_K, msg_bytes, PARAM_K); /* (1) */
    memcpy(cdw, cdw_bytes, PARAM_N1);
}
```

Inspecting this routine confirms the intended systematic structure: the encoder first computes the parity prefix of the RS codeword (omitted above), where the core arithmetic is driven by finite-field multiplications (`gf_mul`) and subsequent shift/XOR updates of `cdw_bytes`. It then appends the information part by directly copying `msg_bytes` into the last `PARAM_K` symbols via the final copy operation (1), i.e., the message bytes are preserved verbatim in the RS codeword.

Concretely, for the analyzed parameter set, the RS codeword has length $\mathtt{PARAM\_N1} = 46$ bytes, while the RS message has length $\mathtt{PARAM\_K} = 16$ bytes (i.e., 128 bits). Due to

(1), after RS encoding, the message bytes appear verbatim in the systematic part of the codeword:

$$\mathsf{cdw\_bytes}[30..45] = \mathsf{msg\_bytes}[0..15],$$

whereas the remaining prefix

$$\mathsf{cdw\_bytes}[0..29]$$

contains the redundancy (parity) symbols computed from msg_bytes.

This systematic placement will be useful in the following analysis because it makes the message-dependent portion of the RS codeword explicit and byte-aligned before RM encoding. In the next subsection, we show that the subsequent RM encoding routine introduces a strong leakage point that enables direct recovery of the message bytes.

After RS encoding, HQC applies the inner RM encoder independently to each RS symbol. Concretely, the array `cdw_bytes`$[0..45]$ is processed byte-by-byte: for each $i \in \{0, \ldots, 45\}$, the encoder takes the 8-bit input `cdw_bytes`$[i]$ and produces a 128-bit RM codeword, so the RM encoding routine is executed a total of 46 times per message encoding.

HQC uses the first-order Reed–Muller code RM$(1, 7)$ with parameters $[128, 8, 64]$, which can be viewed as a linear mapping from an 8-bit vector to a 128-bit codeword. Equivalently, for an input byte interpreted as a length-8 binary vector, encoding corresponds to multiplying by a fixed generator matrix $G \in \mathbb{F}_2^{8\times 128}$, i.e., $\mathbf{c} = \mathbf{m}G$ over $\mathbb{F}_2$.

In the HQC implementation, the generator rows of $G$ are realized via compact 32-bit masks that match the standard RM$(1, 7)$ pattern. The masks are arranged as follows:

```
 * 0   aaaaaaaa aaaaaaaa aaaaaaaa aaaaaaaa
 * 1   cccccccc cccccccc cccccccc cccccccc
 * 2   f0f0f0f0 f0f0f0f0 f0f0f0f0 f0f0f0f0
 * 3   ff00ff00 ff00ff00 ff00ff00 ff00ff00
 * 4   ffff0000 ffff0000 ffff0000 ffff0000
 * 5   ffffffff 00000000 ffffffff 00000000
 * 6   ffffffff ffffffff 00000000 00000000
 * 7   ffffffff ffffffff ffffffff ffffffff
```

Rows 0–4 exhibit a 32-bit periodic structure over the 128-bit codeword, while rows 5–6 have 64-bit periodicity, and row 7 is the all-ones mask. This regular structure enables an efficient Cortex-M4 implementation that avoids explicit bit-matrix multiplication over all 128 output positions.

Rather than computing the $\mathbb{F}_2$-linear combination $\mathbf{m}G$ bit-by-bit, the Cortex-M4 optimized code path uses word-level masking and XORs. The key idea is to expand each message bit into a full-word mask (all-zero or all-one) and then conditionally XOR fixed constants that correspond to generator-row patterns. The resulting routine is shown below.

The last three bits are handled slightly differently: bit 7 initializes an initial pattern, while bits 5 and 6 toggle the full 32-bit word (without an `AND` mask) to realize longer-period (64/128-bit) patterns across the four 32-bit limbs.

For implementation convenience, HQC expands each input bit into an all-zero or all-one 32-bit mask via `BITOMASK(x) = -((x)&1)`. Concretely, if the selected message bit is 0, `BITOMASK` evaluates to `0x00000000`; if the bit is 1, it evaluates to `0xFFFFFFFF`. The encoder then combines these masks with fixed constants such as `0xaaaaaaaa` and `0xcccccccc` using bitwise `AND` and `XOR`. As we show next, this "bit-to-word" expansion leads to a highly regular instruction sequence whose intermediate register values are strongly data dependent.

Listing 2: Reed–Muller encoding routine in HQC (excerpt).

```
#define BIT0MASK(x) (int32_t)(-((x) & 1))

typedef union {
    uint8_t u8[16];  /**< Byte-wise access (16 bytes) */
    uint32_t u32[4]; /**< Word-wise access (4 32-bit words) */
} rm_codeword_t;

void encode(rm_codeword_t *word, int32_t message) {
    int32_t first_word;

    first_word = BIT0MASK(message >> 7);
    first_word ^= BIT0MASK(message >> 0) & 0xaaaaaaaa;
    first_word ^= BIT0MASK(message >> 1) & 0xcccccccc;
    first_word ^= BIT0MASK(message >> 2) & 0xf0f0f0f0;
    first_word ^= BIT0MASK(message >> 3) & 0xff00ff00;
    first_word ^= BIT0MASK(message >> 4) & 0xffff0000;
    word->u32[0] = first_word;

    first_word ^= BIT0MASK(message >> 5);
    word->u32[1] = first_word;
    first_word ^= BIT0MASK(message >> 6);
    word->u32[3] = first_word;
    first_word ^= BIT0MASK(message >> 5);
    word->u32[2] = first_word;
    return;
}
```

We target the `CW308_STM32F3` platform and compile the RM encoder with `arm-none-eabi-gcc` using `-O3`. Under these settings, the RM encoding routine produces the following Cortex-M4 assembly excerpt (one RS symbol per iteration):

The key observation is that the compiler realizes `BIT0MASK(message » i)` using `SBFX` (Signed Bit-Field Extract). Specifically, `sbfx rX, r3, #i, #1` extracts the $i$-th bit of the byte in `r3` and sign-extends it to a 32-bit value. Therefore, the output register becomes either `0x00000000` (if the extracted bit is 0) or `0xFFFFFFFF` (if the bit is 1), matching exactly the semantics of `BIT0MASK`. The subsequent `AND` operations with constants (`0xaaaaaaaa`, `0xcccccccc`, `0xf0f0f0f0`, `0xff00ff00`, `0xffff0000`) select large, highly structured bit patterns depending on the message bit, and the `EOR` chain xors these patterns into the evolving accumulator.

Listing 3: Cortex-M4 `-O3` assembly excerpt for RM encoding (excerpt; abridged).

```
080005c4 <encode>:
 80005c4: f341 0240 sbfx r2, r1, #1, #1
 80005c8: f341 13c0 sbfx r3, r1, #7, #1
 80005cc: f002 32cc and.w r2, r2, #0xcccccccc
 80005d0: 4053 eors r3, r2
 80005d2: f341 0200 sbfx r2, r1, #0, #1
 80005d6: f002 32aa and.w r2, r2, #0xaaaaaaaa
 80005da: 4053 eors r3, r2

  @ ... (similar sbfx/and/eors sequence for bits #2--#4 omitted) ...

 80005fa: f341 1240 sbfx r2, r1, #5, #1
 80005fe: 6003 str r3, [r0, #0]
 8000600: f341 1180 sbfx r1, r1, #6, #1
 8000604: 4053 eors r3, r2
 8000606: 4059 eors r1, r3
 8000608: 404a eors r2, r1
 800060a: 6043 str r3, [r0, #4]
 800060c: 60c1 str r1, [r0, #12]
 800060e: 6082 str r2, [r0, #8]
 8000610: 4770 bx lr
```

From a leakage-analysis perspective, this instruction sequence is particularly favorable to an attacker: each input bit is expanded into a full 32-bit all-zero or all-one value (`SBFX` output), which in turn drives a cascade of wide-register logical operations. As a result, the Hamming weight and bit transitions of intermediate registers and memory stores become strongly correlated with individual message bits. Since this pattern repeats for each of the 46 RS symbols (and the systematic RS layout fixes which symbols correspond directly to message bytes), the RM encoder provides a highly structured and repeatable source of message-dependent side-channel leakage, enabling straightforward byte- and bit-level recovery in the following sections.

## 3.2 Empirical Leakage Assessment of RM Encoding

To quantify the practical leakage of the RM-encoding routine, we collected power traces using a PicoScope 3403D and a ChipWhisperer CW308 UFO board equipped with an STM32F303 target board. A ChipWhisperer-Lite was connected in parallel to provide precise trigger generation and to control the execution flow (reset/arming), ensuring consistent trace alignment across measurements.

All measurements in this section target the RM encoder in the official HQC upstream code base ([Tea26]). We compile the implementation with `-Os` and profile only the `rm_encode` routine in isolation (i.e., excluding the rest of the decapsulation/re-encryption path) to characterize whether the implementation exhibits clear message-dependent leakage. In this subsection, we focus solely on *detecting* and quantifying leakage (e.g., via standard leakage tests); message inference and single-trace recovery are treated separately in the subsequent sections.

After establishing the measurement setup, we performed a first-order leakage test using a two-sample Welch's t-test with a bit-conditioned fixed-vs-fixed (FvF) partitioning. Concretely, we feed `rm_encode` with a single input byte and, for each target bit position $b \in \{0, \ldots, 7\}$, form two trace sets: (i) a set where bit $b$ is fixed to 0 while the remaining seven bits are sampled uniformly at random, and (ii) a set where bit $b$ is fixed to 1 while the remaining bits remain random. This isolates the contribution of the selected bit while averaging out effects from the other input bits.

For each bit position, we collected 200 traces per set (400 traces total), and computed both the mean power difference between the two sets and the corresponding Welch $t$-statistic at each time sample. We use TVLA (Test Vector Leakage Assessment) in the sense of Gilbert *et al.* [GGJR+11] to localize leakage points statistically: time samples where $|t| > 4.5$ are typically considered evidence of statistically significant first-order leakage. We report these results per bit to reveal which parts of the RM-encoding instruction sequence exhibit strong, temporally localized data dependence.

**Results and interpretation.** Figure 2 shows the resulting $t$-test traces (overlay across the 16 iterations, together with the mean). We observe strong, repeatable leakage points whose locations match the compiler-generated instruction sequence. In particular, the POIs appear in the same order as the bit-processing order in the RM encoder of Listing 2: the routine effectively handles bits in the sequence $7 \to 0 \to 1 \to 2 \to 3 \to 4 \to 5 \to 6$, and the $t$-test peaks arise accordingly.

Moreover, the mean power difference curve closely aligns with the locations where the Welch $t$ statistic exceeds the detection threshold, providing consistent evidence of message-dependent leakage at the same time offsets.

This behavior indicates that leakage associated with each message bit manifests at *distinct* and largely non-overlapping time offsets. Such temporal separation is advantageous to an attacker, as it enables independent per-bit discrimination with minimal interference from other bits.

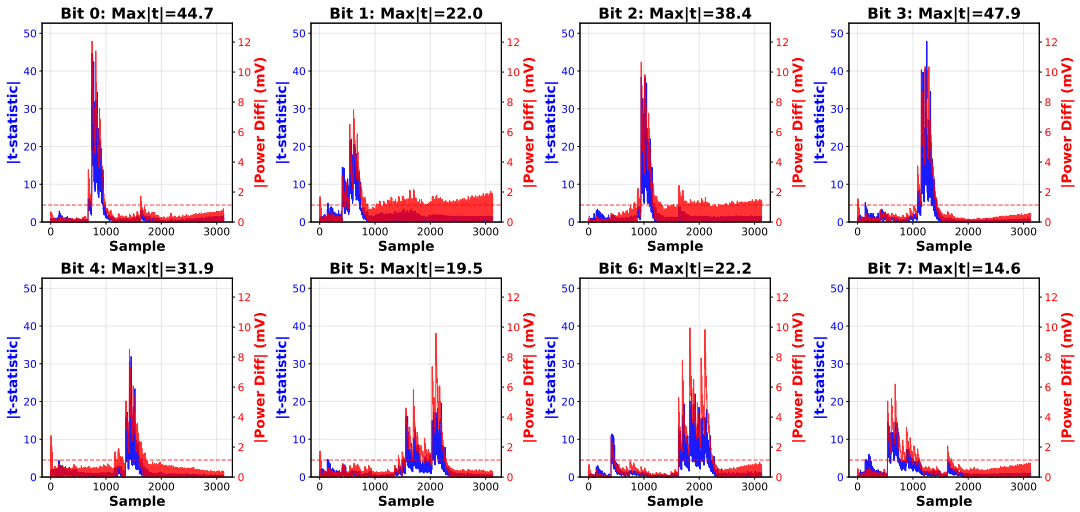


**Figure 2:** TVLA of RM encoding for one input byte (400 traces/bit: 200 with bit=0 and 200 with bit=1; other bits random). Blue/left: Welch $t$ values; green/right: mean power difference.

**Table 1:** Per-bit MLP accuracy on RM-encoding traces (320 train / 80 test per bit).

| **Bit** | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | **Avg.** |
|---|---|---|---|---|---|---|---|---|---|
| **Accuracy (%)** | 97.50 | 98.75 | 98.75 | 96.25 | 98.75 | 96.25 | 98.75 | 97.50 | **97.81** |

To confirm that the identified POIs are directly exploitable, we performed a simple profiling experiment using a lightweight MLP classifier per bit. Using 320 traces for training and 80 traces for testing (per bit), we obtain consistently high classification accuracy across all eight bits, as summarized in Table 1.

These results indicate that each message bit can be distinguished reliably from the RM-encoding leakage with only a few hundred profiling traces per bit, confirming that the leakage is strong and readily exploitable.

# 4 From Message Leakage to Full Secret-Key Recovery

Recent work has demonstrated that full secret-key recovery against HQC becomes practical once an attacker can instantiate a *decryption-related oracle* from side-channel leakage and then combine oracle outputs with an optimized post-processing step. Two representative frameworks are OT-PCA [DG25b] and its extended version that constructs richer oracle models, namely multi-value plaintext-checking (MV-PC) and full-decryption (FD) oracles [DG25a].

In the introduction of [DG25a], the authors summarize the evolution of plaintext-checking and decryption-failure oracles and emphasize the following point.

> They show that the initial mathematical breakthrough in exploiting an oracle is the critical enabler; once this theoretical foundation is established, the research challenge shifts to discovering physical leakages on diverse platforms to instantiate the required oracle.

This statement highlights that, after the oracle-based key-recovery methodology is theoretically well established, the main remaining bottleneck is often *oracle instantiation*

*at the implementation level*—i.e., extracting oracle outputs reliably and efficiently from real side-channel leakage.

From this perspective, HQC oracle-based key recovery can be considered theoretically mature, with MV/FD-oracle models providing a strong and well-founded attack framework. Accordingly, the next step is to optimize the *implementation-level* realization of such oracles.

In this chapter, we discuss how our *single-trace message recovery* from RM-encoding leakage can be lifted into an oracle primitive, and how this affects the practicality of oracle construction. In particular, we evaluate how efficiently PC/MV-PC/FD-style oracle behavior can be instantiated when the decrypted message is recovered directly from RM-encoding leakage, and we compare the resulting costs to the offline-template-based instantiations in [DG25b, DG25a].

## 4.1 Recap of OT-PCA: Oracle Crafting and Offline Templates

OT-PCA [DG25b] is an offline-template-based key-recovery framework that builds on the oracle-crafting idea of Schröder *et al.* [SGG24]. Its key insight is that a 1-bit plaintext-checking (PC) outcome can be made highly *key-dependent* through chosen-ciphertext structure, and that offline profiling can turn these noisy PC observations into *position-wise soft information* on the sparse secret.

OT-PCA starts from the HQC-PKE relation

$$\mathbf{u} = \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2, \qquad \mathbf{v} = \mathcal{C}.\mathsf{Encode}(m) + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e},$$

and decryption

$$m \leftarrow \mathcal{C}.\mathsf{Decode}\big(\mathbf{v} - \mathbf{u} \cdot \mathbf{y}\big).$$

Following [SGG24, DG25b], the attacker enforces simple monomial forms for $(\mathbf{r}_1, \mathbf{r}_2)$ to isolate a secret-dependent term in the decoder input. For example, setting $\mathbf{r}_1 = X^k$ and $\mathbf{r}_2 = \mathbf{0}$ yields

$$\mathbf{u} = X^k, \qquad \mathbf{v} = m\mathbf{G} + \mathbf{e},$$

so the decoder sees

$$\mathbf{v} - \mathbf{u} \cdot \mathbf{y} = m\mathbf{G} + \mathbf{e} - X^k \cdot \mathbf{y}.$$

Alternatively, setting $\mathbf{r}_1 = \mathbf{0}$ and $\mathbf{r}_2 = X^k$ gives

$$\mathbf{v} - \mathbf{u} \cdot \mathbf{y} = m\mathbf{G} + \mathbf{x} \cdot X^k + \mathbf{e},$$

using $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$. In both cases, the PC outcome (accept/reject) becomes sensitive to the interaction between the crafted offset $X^k$, the fixed error pattern $\mathbf{e}$, and the secret.

A crucial practical step is selecting $\mathbf{e}$ so that the PC bit is informative: $\mathbf{e}$ is tuned to place the decoder near its correction boundary, increasing the entropy of the accept/reject outcome and improving distinguishability across targeted positions [DG25b]. Finally, OT-PCA trains offline templates that map leakage observations to PC outcomes and converts these 1-bit observations into position-wise soft information used by the subsequent key-recovery stage.

The later MV-PC and FD frameworks [DG25a] follow the same overall pipeline (chosen-ciphertext crafting, $\mathbf{e}$ optimization, and offline templates), but enlarge the oracle output space from a binary PC bit to multi-valued labels to extract more information per trace.

## 4.2 MV-PC and FD Oracles from Offline Templates

**MV-PC oracle (multi-valued outcomes from FO re-encryption).** In the multi-value plaintext-checking (MV-PC) setting [DG25a], a single crafted ciphertext family

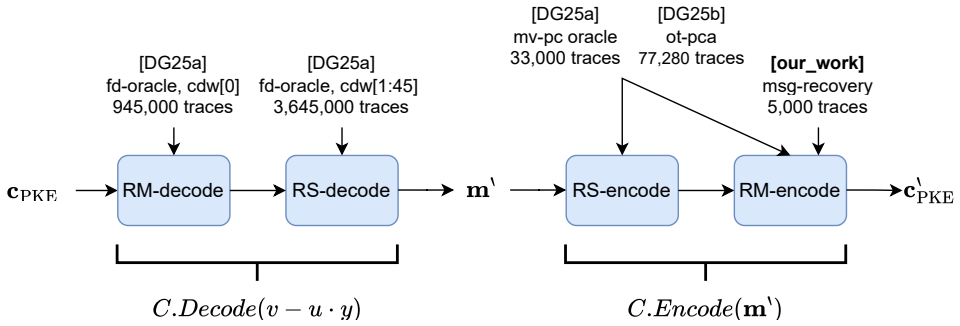


**Figure 3:** Oracle instantiation targets and profiling costs in prior work and our work.

(parameterized by $k$ and a fixed error pattern $\mathbf{e}$) can decrypt to different candidate messages $m'$ depending on the secret key. MV-PC exploits the FO structure of HQC-KEM, where the decrypted message $m'$ is deterministically reused in the re-encryption step. Instead of a binary accept/reject bit, the attacker therefore aims to classify $m'$ (or a label derived from $m'$) from side-channel leakage in the re-encryption path. Accordingly, the offline objective for choosing $\mathbf{e}$ shifts from maximizing the entropy of a single PC bit to maximizing the diversity/entropy of the resulting $m'$ distribution, enabling higher-information oracle outputs per trace.

**FD oracle (decoding intermediates before RS correction).** The full-decryption (FD) oracle further increases information per query by targeting intermediate decoder states rather than the final decoded message [DG25a]. Since HQC decoding is concatenated (RM decoding followed by RS decoding), the RS stage may correct symbol errors and thereby merge multiple distinct RM-level outcomes into the same final message, reducing observable information. FD therefore targets the RM-decoding stage directly and maps leakage to RM-level outcomes (or derived labels) *prior* to RS correction. As in OT-PCA and MV-PC, an offline phase selects $\mathbf{e}$ to yield a high-entropy outcome distribution, and offline templates are trained to classify these multi-valued FD outcomes from leakage traces.

## 4.3 Summary of Oracle Instantiation Costs in Prior Work

Figure 3 summarizes *where* each SOTA oracle is instantiated along the HQC decapsulation flow (RM-decode, RS-decode, RS-encode, RM-encode) and the corresponding profiling trace budgets reported in prior work, together with our RM-encoding-based message-recovery oracle.

**OT-PCA (PC oracle, 2 classes).** OT-PCA [DG25b] instantiates a *binary* plaintext-checking oracle whose output is mapped to two classes (accept/reject). To train the offline template for this 1-bit oracle, the authors collect 51,520 profiling traces to learn the mapping for a fixed message labeling (0/1), which is later converted into position-wise soft information.

**MV-PC oracle (11 classes from $m'$).** In the MV-PC setting of [DG25a], a fixed crafted error pattern can yield multiple possible decrypted messages $m'$ depending on the secret. The authors report observing 11 distinct $m'$ outcomes for their chosen setup, and they train a multi-class offline template by collecting 3,000 traces per class, for a total of $11 \times 3{,}000 = 33{,}000$ profiling traces.

**FD oracle (27 classes from decoding intermediates).** For the FD oracle of [DG25a], the outcome alphabet is further enlarged by targeting decoding intermediates (to avoid the information loss induced by RS correction). Concretely, for tracking `cdw[0]`, the template targets the RM-decoding stage and collects 35,000 traces per class across 27 classes, totaling $27 \times 35{,}000 = 945{,}000$ traces. For the remaining `cdw[1..45]`, the authors target leakage in the RS-decoding stage and report collecting 3,000 traces per class for 27 classes across 45 positions, totaling $45 \times 27 \times 3{,}000 = 3{,}645{,}000$ traces.

**Implication for our oracle.** By contrast, our oracle recovers the decrypted message $m'$ directly from RM-encoding leakage. Since $m'$ is the value deterministically reused by the FO re-encryption during decapsulation, this message-recovery primitive can be *lifted* into prior oracle models with minimal additional assumptions.

Both OT-PCA's PC oracle [DG25b] and the MV-PC oracle of [DG25a] ultimately define their oracle output as a function of the decrypted message $m'$: the PC oracle evaluates a binary predicate (accept/reject), while MV-PC assigns a multi-valued label derived from $m'$. Therefore, once $m'$ is recovered, the attacker can compute the corresponding PC/MV-PC oracle outputs *locally* without building class-specific multi-class templates, i.e., our primitive can *fully replace* the oracle instantiation step in both settings.

For the FD setting [DG25a], our primitive enables a *partial replacement*. FD oracles target decoding intermediates (e.g., RM-level symbol decisions) to avoid the information loss induced by the subsequent RS correction step (cf. Section 2.2). While our leakage source sits on the *encoding* side and does not directly expose all FD-specific intermediate labels, we can still cover a meaningful subset of FD targets by *forcing RS decoding failure* and exploiting the *systematic* RS layout (Section 2.2).

Concretely, HQC-128 uses a shortened systematic RS code with parameters $(n_1, k, d_{\min}) = (46, 16, 31)$, hence a symbol-error correcting capacity of $\delta = (n_1 - k)/2 = 15$ (Section 2.2). By crafting ciphertexts that inject more than $\delta$ erroneous RS symbols into the *parity region* (`cdw[0..29]`) while keeping the *systematic message region* (`cdw[30..45]`) intact, the attacker can push RS decoding beyond its correction capability so that RS correction no longer succeeds. In this regime, the systematic message symbols are no longer "pulled" toward a corrected RS codeword; instead, the values observed on `cdw[30..45]` correspond essentially to the *RM-decoded* symbol outputs (up to the residual channel noise), yielding an FD-like view for these 16 positions without requiring FD-style multi-class templates for the message part (cf. Section 2.2 for the RM→RS decoding order and the systematic placement).

Operationally, this comes at the cost of an additional crafted-oracle query per targeted instance, but it allows our message-recovery primitive to replace the FD pipeline components that would otherwise be devoted to classifying the message-part outcomes, while the remaining parity-related and FD-specific intermediate labels may still require the original dedicated leakage targets and templates.

# 5 Experimental Results

## 5.1 Experimental Setup

Our experiments use the same measurement platform and acquisition procedure as in the TVLA study of Section 3.2: a ChipWhisperer CW308 UFO board with an STM32F303 target, power traces captured by a PicoScope 3403D at 250 MS/s, and a ChipWhisperer-Lite used for synchronized triggering and execution control (reset/arming) to ensure stable trace alignment.

In Section 3.2, we profiled `rm_encode` in isolation on a single input byte to detect and localize leakage (under `-Os`). In contrast, the experiments in this section cover the *full*

**Table 2:** Sample indices of the RS start trigger and the first RM-encoding start (`rm_encode`) in encapsulation traces

| Trace | RS-encode start | 1st RM-encode start | Diff. (RM–RS) |
|---|---|---|---|
| 0 | 500 | 7265934 | 7265434 |
| 1 | 105057 | 7370491 | 7265434 |
| 2 | 500 | 7265935 | 7265435 |
| 3 | 500 | 7265934 | 7265434 |
| 4 | 500 | 7265934 | 7265434 |
| 5 | 500 | 7265934 | 7265434 |
| 6 | 500 | 7265933 | 7265433 |
| 7 | 500 | 7265934 | 7265434 |
| 8 | 500 | 7265933 | 7265433 |
| 9 | 500 | 7265933 | 7265433 |

RM-encoding process as it occurs in the attack pipeline, i.e., RM encoding is executed over all RS symbols rather than a single byte. Concretely, the target runs the HQC-128 reference implementation compiled with `arm-none-eabi-gcc v.10.3.1` at `-O3`, and we collect 5,000 profiling traces with uniformly random 128-bit messages.

**Trace alignment.** For acquisition efficiency, we trigger on the HQC-PKE.Encryption flow and extract the RM-encoding window offline. We verified that the offset from the RS-encoding start to the first `rm_encode` call is effectively constant across encapsulation traces (Table 2), so a fixed sample shift suffices.

## 5.2 RM-Encoding Message-Recovery Experiment and Results

**Dataset, model, and evaluation.** We collected a total of 5,000 traces, consisting of 4,000 profiling traces for training and 1,000 traces reserved for evaluation. The input messages are uniformly random 128-bit values, and we train a single RM-byte estimator (internally consisting of eight bit-wise classifiers). Each bit classifier is implemented as a 1D CNN, and we improve robustness using a 5-model ensemble whose outputs are combined (e.g., by majority vote or by averaging posterior probabilities). We evaluate performance as a function of the number of traces used per decision, and report three metrics: (i) bit accuracy (averaged over all bits), (ii) byte accuracy (all 8 bits correct for a byte), and (iii) full-message accuracy (all 16 bytes correct for a 128-bit message).

**Results.** Table 3 summarizes the message-recovery performance. With 3,200 traces, we reach 96.9% success in recovering the full 128-bit message. Past this point, the full-message rate remains broadly similar with small fluctuations, which is consistent with diminishing returns (and potentially mild overfitting) once the per-bit and per-byte accuracies are already saturated. Note that the full-message metric is stringent: it counts a success only when *all* 128 bits (i.e., all 16 bytes) are recovered correctly. Hence, even tiny residual bit/byte error rates can limit further improvements when aggregated over the entire message.

**Tables and comparison summary.** Table 4 places our attack in context by comparing it with the SASCA-based single-trace attack of Goy *et al.* [GMGL24]. Both attacks operate in a single-trace setting, but they target different implementation leakage sources and use different inference mechanisms. The SASCA attack models Hamming-weight leakage from RS encoder/decoder operations (notably `gf_mul`) and performs probabilistic inference on a factor graph via belief propagation, requiring 300,000 profiling traces for template construction. In contrast, our approach targets the RM encoder leakage induced by

**Table 3:** Message recovery performance from RM-encoding leakage using bit-wise CNN classifiers (8 binary CNNs per byte). Results are shown for selected trace budgets. Bit and byte accuracies are arithmetic means over 128 bits and 16 bytes, respectively.

| Traces | Bit acc. (%) | Byte acc. (%) | Full msg acc. (%) |
|---|---|---|---|
| 100 | 94.61 | 78.86 | 3.80 |
| 300 | 96.01 | 86.36 | 13.90 |
| 600 | 97.64 | 84.72 | 9.20 |
| 800 | 97.79 | 88.59 | 20.30 |
| 900 | 99.61 | 98.62 | 90.80 |
| 1000 | 99.60 | 98.73 | 92.10 |
| 2000 | 99.50 | 98.87 | 93.90 |
| 3000 | 99.51 | 98.96 | 95.40 |
| 3200 | 99.52 | 99.06 | 96.90 |
| 4000 | 99.52 | 99.00 | 96.00 |

**Table 4:** Comparison with the SASCA-based single-trace attack on HQC [GMGL24].

| | **SASCA** [GMGL24] | **This work** |
|---|---|---|
| Target operation | RS encoder/decoder (`gf_mul`) | RM encoder (`BITOMASK`) |
| Leakage model | Hamming weight | Bit-level (per-bit separation) |
| Analysis method | Factor graph + belief propagation | 8 binary CNN classifiers |
| Implementation | HQC reference (April 2023) | HQC reference (GitLab latest) |
| Compiler / Optimization | `-O3` | `arm-none-eabi-gcc -O3` |
| Target platform | STM32F407 (Cortex-M4) | STM32F303 (Cortex-M4) |
| Training(profiled) | 300,000 (of 500,000) | **3,200** (of 5,000) |
| Trace reduction | — | **≈99%** |
| Success rate | Decoder: 100%; Encoder: 65–81% | Full 128-bit message: **96.9%** |

`BITOMASK`-style bit-to-word expansion, leverages per-bit temporal separation, and uses eight binary CNN classifiers for message recovery. Under our experimental setup, this yields comparable single-trace capability with a substantially smaller profiling budget: 3,200 traces (a reduction of roughly 99%) while recovering the full 128-bit message with 96.9% success.

**Oracle-cost impact.** Table 5 summarizes how direct recovery of the decrypted message $m'$ from RM-encoding leakage affects the *oracle-instantiation* cost of prior HQC key-recovery frameworks. In OT-PCA and MV-PC, the oracle output is ultimately a function of $m'$ (a binary PC predicate for OT-PCA, and a message-derived label for MV-PC). Once $m'$ is recovered, these labels can be computed locally, so the multi-class template training required to *instantiate* the oracle collapses to the profiling budget of our message-recovery primitive (5,000 traces). For FD, our primitive does not replace all intermediate-label templates, but it removes the need to model the systematic message region via dedicated FD templates, reducing the overall profiling burden (from 4.5M to ≈3.3M traces). *Equivalently, for 16 out of 46 FD target blocks, our primitive replaces* $\approx 16 \times 27 \times 3{,}000 \approx 1.30M$ *traces of multi-class templates by only 5,000 traces, meaning that roughly a* $16/46$ *fraction of*

**Table 5:** Oracle instantiation cost with and without our RM-encoding message-recovery primitive. Parentheses report the cost *after* applying our primitive.

| Framework | #Cls | Profiling traces | Trace red. |
|---|---|---|---|
| OT-PCA [DG25b] | 2 | 51,520 (**5,000**) | **90.3%** |
| MV-PC [DG25a] | 11 | 33,000 (**5,000**) | **84.83%** |
| FD [DG25a] | 27 | 4.5M (≈**3.3M**) | ≈**26.7%** |
| **This work** | $2^{128}$ | **5,000** | – |

*Class definition.* #Cls denotes the cardinality of the oracle output alphabet used by each framework: OT-PCA uses a binary plaintext-checking outcome, MV-PC maps leakage to one of 11 message-derived labels, and FD maps leakage to one of 27 decoding-intermediate labels. Our primitive instead recovers the full decrypted message $m' \in \{0,1\}^{128}$ (output space size $2^{128}$), allowing PC/MV-PC labels to be computed locally and partially reducing FD template needs.

*the FD-position-dependent evidence can be obtained with our profiling budget.* Overall, the main gain is that the oracle output space becomes effectively the full $2^{128}$ message domain, avoiding reliance on a small, pre-enumerated class set and substantially lowering the profiling effort needed to realize PC/MV-PC-style oracles in practice.

# 6 Conclusion

HQC has now been selected for standardization, making implementation security on embedded targets a first-order requirement. In this work, we showed that the RM$(1, 7)$ encoding routine constitutes a *systematic* and *cross-implementation* leakage point: the optimized encoder's bit-to-word expansion (e.g., `BITOMASK` or compiler-emitted `sbfx`) yields strong, structured, and temporally separable message-dependent leakage. Exploiting this structure, we demonstrated that recovering the full 128-bit message from a *single* decapsulation trace becomes practical with a small profiling budget, directly endangering session-key confidentiality under FO-style decapsulation.

Beyond message recovery, our results also tighten the link between message leakage and oracle-based attacks on HQC: once the decrypted message is recovered, PC/MV-PC outcomes can be computed locally and parts of FD-style oracle instantiation can be simplified. This perspective aligns with recent oracle-based HQC work, which notes that SASCA-style message recovery can serve as an alternative route to oracle construction and may be adapted toward FD-oracle realization (cf. Section 5.4 of [DG25a]).

# A Classifier Configuration (MLP & CNN)

## A.1 MLP for Bit-wise Leakage Classification

We train an independent binary MLP per bit on POIs extracted from RM-encoding traces. We use an 80/20 split (320/80 traces per bit) with early stopping.

**Table 6:** Per-bit MLP hyperparameters (Table 1).

| Parameter | Value |
|---|---|
| `hidden_layer_sizes` | (64, 32) |
| `activation` | ReLU |
| `max_iter` | 500 |
| `early_stopping` | True (`validation_fraction`=0.15) |
| `random_state` | 42 |

## A.2 CNN Ensemble for Bit-wise Message Recovery

We use eight binary 1D-CNNs per byte and an ensemble of $M = 5$ models. Inputs are standardized (fit on training) and aggregated by averaging softmax posteriors.

**Table 7:** CNN architectures (summary) and training hyperparameters.

| Component | Setting |
|---|---|
| Architectures | `CNN1D`: Conv(32,k=5)→MP2 → Conv(64,k=5)→MP2 → Conv(128,k=3) →AAP1 → FC(128)→2<br>`CNN1DSmall`: Conv(16,k=7)→MP2 → Conv(32,k=5) →AAP1 → FC(64)→2 |
| Loss / Optimizer | CrossEntropy (2-class logits) / AdamW |
| Learning rate / Weight decay | $1 \times 10^{-3}$ / $1 \times 10^{-4}$ |
| Training budget | Max 30 epochs, early stopping patience 5 (val. acc.) |
| Batching | Train batch 256, inference batch 512 |
| Ensemble / Split | $M = 5$ models; 20% stratified validation (`random_state`=42) |

# References


[AAC+22] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the third round of the nist post-quantum cryptography standardization process. 2022.

[ABB+22] Nicolas Aragon, Paulo Barreto, Slim Bettaieb, Loic Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Santosh Ghosh, Shay Gueron, Tim Güneysu, et al. Bike: bit flipping key encapsulation. 2022.

[ABC+25] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, et al. Status report on the fourth round of the nist post-quantum cryptography standardization process, 2025.

[BCL+17] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic mceliece: conservative code-based cryptography. *NIST submissions*, 1(1):1–25, 2017.

[BDK+18] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018.

[Ber25] Pierre-Augustin Berthet. Indepth analysis of a side-channel message recovery attack against frodokem. In *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 666–671. IEEE, 2025.

[BHK+19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.

[BL17] Daniel J Bernstein and Tanja Lange. Post-quantum cryptography. *Nature*, 549(7671):188–194, 2017.

[CCJ+16] Lily Chen, Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray A Perlner, and Daniel Smith-Tone. *Report on post-quantum cryptography*, volume 12. US Department of Commerce, National Institute of Standards and Technology . . . , 2016.

[CDCG22] Brice Colombier, Vlad-Florin Drăgoi, Pierre-Louis Cayrel, and Vincent Grosso. Profiled side-channel attack on cryptosystems based on the binary syndrome decoding problem. *IEEE Transactions on Information Forensics and Security*, 17:3407–3420, 2022.

[DG25a] Haiyue Dong and Qian Guo. Multi-value plaintext-checking and full-decryption oracle-based attacks on hqc from offline templates. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):254–289, 2025.

[DG25b] Haiyue Dong and Qian Guo. Ot-pca: New key-recovery plaintext-checking oracle based side-channel attacks on hqc with offline templates. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):251–274, 2025.

[DKL+18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.

[FHK+18] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang, et al. Falcon: Fast-fourier lattice-based compact signatures over ntru. *Submission to the NIST's post-quantum cryptography standardization process*, 36(5):1–75, 2018.

[FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptology conference*, pages 537–554. Springer, 1999.

[GAMA+25] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.

[GGJR+11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. A testing methodology for side-channel resistance validation. In *NIST non-invasive attack testing workshop*, volume 7, pages 115–136, 2011.

[GMGL24] Guillaume Goy, Julien Maillard, Philippe Gaborit, and Antoine Loiseau. Single trace hqc shared key recovery with sasca. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):64–87, 2024.

[HSC+23] Senyang Huang, Rui Qi Sim, Chitchanok Chuengsatiansup, Qian Guo, and Thomas Johansson. Cache-timing attack against hqc. *Cryptology ePrint Archive*, 2023.

[LNPS20] Norman Lahr, Ruben Niederhagen, Richard Petri, and Simona Samardjiska. Side channel information set decoding using iterative chunking: Plaintext recovery from the "classic mceliece" hardware reference implementation. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 881–910. Springer, 2020.

[MAB+18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loıc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.

[Mil86] Victor S Miller. Use of elliptic curves in cryptography. *Advances in Cryptology—CRYPTO'85 Proceedings*, pages 417–426, 1986.

[MS77] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error-correcting codes*, volume 16. Elsevier, 1977.

[NDGJ21] Kalle Ngo, Elena Dubrova, Qian Guo, and Thomas Johansson. A side-channel attack on a masked ind-cca secure saber kem implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 676–707, 2021.

[RBRC21] Prasanna Ravi, Shivam Bhasin, Sujoy Sinha Roy, and Anupam Chattopadhyay. On exploiting message leakage in (few) nist pqc candidates for practical message recovery attacks. *IEEE Transactions on Information Forensics and Security*, 17:684–699, 2021.

[RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[SGG24] Robin Leander Schröder, Stefan Gast, and Qian Guo. Divide and surrender: Exploiting variable division instruction timing in {HQC} key recovery attacks. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 6669–6686, 2024.

[Sho94] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.

[SKL+20] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Tae-Ho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. Single-trace attacks on message encoding in lattice-based kems. *IEEE Access*, 8:183175–183191, 2020.

[Tea26] HQC Team. HQC: Official reference implementation (gitlab), commit d622142a50f3ce6b6e1f5b15a5119d96c67194e0. https://gitlab.com/pqc-hqc/hqc/-/commit/d622142a50f3ce6b6e1f5b15a5119d96c67194e0, 2026. Accessed: 2026-01-14.

[VCGS14] Nicolas Veyrat-Charvillon, Benoît Gérard, and François-Xavier Standaert. Soft analytical side-channel attacks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 282–296. Springer, 2014.